Faculty of Science and Technology
DAT535 — Data Engineering Project

DAT535 Project Report:

Kolumbus and Agder Timetable History

A Medallion-architecture pipeline for Entur NeTEx public-transport timetables with Apache Spark

GitHub Repository: github.com/TOGUNOLUWATOBI/dat535-project

Parneet Kaur Dhanoa
University of Stavanger
Stavanger, Norway

Tobi
University of Stavanger
Stavanger, Norway

Data: Entur national NeTEx timetable exports (NLOD licence) — “Data from Entur”
Stavanger, November 2026

Abstract

In this project we built an Apache Spark pipeline for the **planned bus and boat timetables** from Kolumbus in Rogaland (`KOL`) and Agder Kollektivtrafikk (`AKT`). The data comes from Entur's national NeTEx exports, and we used the Medallion architecture. GitHub Actions downloads the exports on a schedule and stores them unchanged in the **Bronze** layer. The **Silver** layer connects the references in the data, converts the times, works out which days each trip runs and marks any bad rows. A quality gate decides if **Gold** can be published. Gold has daily numbers per line and per operator. We also built a route-history part that tracks how lines change, and a Streamlit GUI to look at the results.

Our main evaluation sample is nine historical snapshots (the 15th of January–September 2026). They have 263,757 journeys and 8,296,954 passing times, and Silver kept all of them with no bad rows. A tenth snapshot, the first scheduled export from 8 November 2026, is added in the quality and route-history results. For January–September 2026 we have all 273 days, with **1,918,707 planned departures**. On the school server, the rebuild covered January–November. Route history confirmed 16 renumbered lines and 4 removed lines, including the Bussveien changes in Stavanger. All 175 tests pass.

Keywords: *Apache Spark, Medallion Architecture, Data Engineering, NeTEx, Public Transport, Data Quality*

1 Introduction

1.1 Problem

Bus routes, line numbers, stops and timetables in Rogaland and Agder change during the year, for example in school holidays, in summer, and with big changes like the new Bussveien between Stavanger and Sandnes on 1 July 2026. Entur publishes the current plans every day, but each new export replaces the old one. There is no easy way to see what was published before and what changed. Our project saves these exports so the planned service can be compared over time.

1.2 What the project builds

- An automatic ingestion step that downloads Entur's national export, keeps only the KOL and AKT data and the national stops, and saves it with all its source information (Bronze).
- A PySpark pipeline that turns the nested XML into clean, connected Parquet tables (staging → Silver), and builds daily service numbers (Gold) only if a quality gate passes.
- A route-history part that tracks line changes between exports, and only calls a change a "renumbering" or "discontinued" if the operator has confirmed it.
- A Streamlit GUI, plus checkpoints, tests and GitHub Actions, so the whole thing can be run again by someone else.

1.3 Questions the outputs answer

1. Which lines were added, removed, changed route or got a new number between two exports, and which of these changes did the operator confirm?
2. How many departures are planned in each month, per day and per weekday, and how does this change during the year?
3. For one line on one date, which version of the line is used and how many departures are planned?

1.4 Consumer and scope

One important point: our results show the **published plans**, not what the buses actually did. If two snapshots are different, it only means the published data changed between those dates.

Our users are transport analysts or journalists who want to know what changed in the network and how sure we are about it. They can use the GUI, the route-history command (`scripts/run.sh inspect`) and the quality reports.

2 Background and Dataset

Entur is the national public transport data service in Norway. It publishes all planned timetables in NeTEx, which is the European standard for timetable data [2]. The data is open under the Norwegian open data licence (NLOD) [3]. We used the **Medallion architecture** [4]: Bronze keeps the raw data, Silver has cleaned data, and Gold has summary tables ready to use. We built it with Apache Spark [1] and run it with GitHub Actions on our own runner [5].

2.1 Source, format and volume

Entur publishes one national ZIP file with NeTEx XML every day, about 280 MB. In our test on 24 October 2026 we kept 532 XML files (322 for Kolumbus, 209 for AKT and the national stops file). We saved them in a 63 MB ZIP and checked all 1,296,889 references with no errors. The whole ingestion took 43.5 seconds.

The XML files are very nested. A `CompositeFrame` has a `ServiceFrame` (lines, routes, stop patterns and stops), a `TimetableFrame` (trips with their times) and a `ServiceCalendarFrame` (which days the trips run). The physical stops (`NSR:StopPlace` and `NSR:Quay`) are in a separate national stops file.

2.2 Why the data suits distributed processing

Just the nine historical snapshots have 263,757 journeys and 8,296,954 passing times. In Silver we need big joins (passing times to stops and calendars) for every snapshot, and we have to run everything again when we change a rule. Spark can read the XML with a schema, do the joins in parallel and save checkpoints. With pandas on one machine we would have had to split the data by hand.

2.3 Characteristics of the data

- Each export only covers a certain period. Most 15 January files start on 12–13 January, so one export

cannot cover a whole month. We needed extra exports for the first days.

- Some exports are broken. The 1 and 2 February 2026 exports have a missing stop (`NSR:Quay:43289`), so we used the 14 January export, which also covers February.
- AKT uses the same ID and version in more than one file (65 times in the October exports). We keep these as warnings and do not merge them.
- Times after midnight are written with a day offset (for example `00:10:00` + 1 day), and on-demand buses (HentMeg) have a time window instead of a fixed time.
- Some files dated 2025 were exactly the same as later files, so we left them out.

2.4 Access, licensing and reproducibility

The data is open and does not need an API key. Every Bronze manifest saves the source link, size and checksums, so if we delete a national ZIP we can download it again and check that it is the same file.

3 Method

In this section we explain the pipeline one layer at a time. All the code is in our GitHub repository. We only show the most important parts here, copied from our code.

3.1 Architecture Overview

Our pipeline uses the Medallion architecture. We added a staging step between Bronze and Silver, and a route-history step after Gold. Every Spark step is saved as a checkpoint in a small SQLite database. The actual data is only stored in Parquet files.

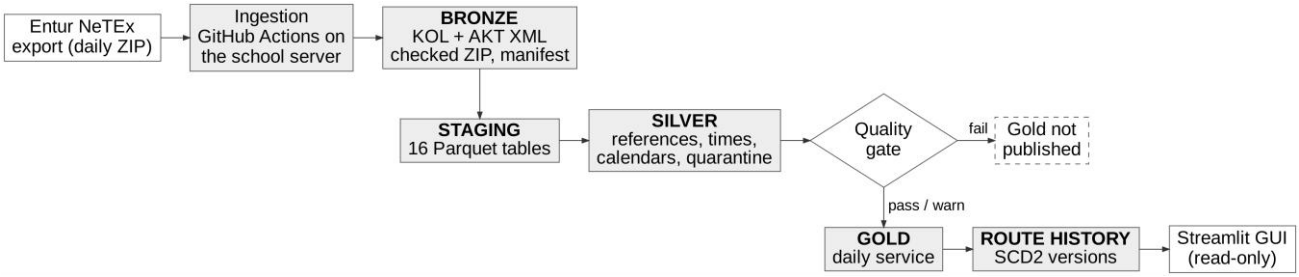


Figure 1. End-to-end architecture of the pipeline

3.2 Ingestion and Bronze Layer

Ingestion copies each Entur export to the school server, checks it and records where it came from. The result is Bronze, a raw copy that we never change, so we can always build the other layers again from it. The ingestion code only uses normal Python, so it runs without Spark and its tests are quick.

3.2.1 Four ways data enters Bronze

- **Historical:** nine exports from the 15th of each month, January–September 2026.
- **Scheduled:** GitHub Actions downloads a new export on days 1, 4, 7 and so on of each month.
- **Manual ZIP:** we can import a ZIP file we already have.
- **Calendar support:** extra exports from the start of each month, to cover the first days.

All four use the same code to unzip, select and check the files. The next steps only accept a Bronze run that ended as `complete` or `complete_with_warnings`.

3.2.2 Scheduled ingestion on the school server

GitHub Actions [5] starts the job and sends it to our **self-hosted runner** on the school server. The server runs the Python code and saves the data on its own disk. The schedule `23 8 */3 * *` starts a run at 08:23 UTC on day 1, 4, 7 and so on, and every push to `main` also starts a run. Each run downloads **yesterday's** export (Oslo time), so we never get a half-finished export.

Before anything else, the workflow checks the Python version, that the data folder exists and can be written to, that it is outside the GitHub checkout folder (so it cannot be deleted by mistake), and that there is at least 2 GiB free. If any check fails, the job stops before downloading.

3.2.3 What happens in one ingestion run

1. **Run tests first.** The ingestion tests run on the server before we download anything.
2. **Get the file info.** Ask Entur for the export of one date and save its size, MD5 and version.
3. **Download.** Each download gets its own folder, so a broken download is kept and not overwritten.
4. **Check the ZIP.** Compare the size, MD5 and SHA-256 with the file info, and check the CRC of every file in the ZIP.
5. **Pick our operators.** Keep the `KOL_*` and `AKT_*` files, their shared files and the national `_stops.xml`. Ignore all other operators.
6. **Check the XML.** Read every file, check the dates and check that every reference points to something that exists. A missing reference stops the run. Repeated IDs are saved as warnings.
7. **Save as a small ZIP.** Put the selected XML files into one ZIP, then read it back and check every file again.
8. **Clean up.** Delete the big 280 MB national ZIP only if steps 4–7 all passed. If something failed, we keep everything.
9. **Finish.** Write the manifest, lineage and validation files, link duplicate files, and upload a copy of the metadata to GitHub. Then processing starts (Sections 3.3–3.4).

3.2.4 Retries, late data and repeated runs

- If there is a timeout or a server error (HTTP 408, 429 or 5xx), we try **three times** and wait a bit between tries.
- If a date is not found (**404**), the run fails. We never pick another date by ourselves, so a missing day is easy to see.
- If a checksum or check fails, the run stops, the manifest is marked `failed` and the GitHub job fails. All files are kept.
- Every run gets a new folder called `run-RUN_ID-ATTEMPT`. Writing to a folder that already exists is not allowed (`FileExistsError`), so nothing is ever overwritten.
- Each run gets an `ingestion_id` based on its content, so we can see when two runs downloaded the same data. Files that are exactly the same are linked instead of copied. One clean-up like this saved 1,957,911,441 bytes.

3.2.5 Manual ZIP ingestion and duplicate uploads

`ingest_local_zip` lets us import a ZIP file we already have, without internet. It copies the file to `uploads/original.zip`, saves its checksums and size, checks every file in the ZIP, and then does the same checks as the scheduled ingestion. The user has to give the date of the export.

To find duplicates we use **the ZIP's SHA-256 + the date + the operators**, saved in a small SQLite file (`_manual_ingestions.sqlite3`). Renaming the file does not trick it. If the same ZIP is uploaded again, we only save a small `duplicate` record that points to the first one, and no second copy is stored. A lock makes sure two uploads at the same time cannot both be saved. We have 49 ingestion tests for cases like this.

3.2.6 Calendar-support ingestion

Most 15 January files only start on 12–13 January, so one mid-month export does not cover the whole month. `prepare_monthly_support` gets and checks an export from the start of each month. If that export is broken, it tries an earlier one that covers the whole month. The 1 and 2 February 2026 exports were broken, so we kept them and used the 14 January export instead. These extra exports are marked `calendar_support`, so they fill in daily numbers but are not counted as extra snapshots in route history.

3.2.7 Storage decision: selected ZIPs instead of expanded XML

The school VM only has a 38 GiB disk. With all the XML unzipped (mostly the national stops file), Bronze grew to 20–23 GiB and only 3.5 GiB was free. Removing duplicate files helped for a short time. The real fix was to keep each snapshot's XML **inside a checked ZIP file** and only unzip it while that snapshot is being processed.

For the 15 January snapshot, 820 MB of XML became a 63 MB ZIP. After we changed all snapshots on the server, the free space went from 3.5 GiB to 23 GiB. The script checks every file before it deletes anything, and when we process a snapshot it is unzipped to the same place and cleaned up afterwards.

3.2.8 From Bronze to staging Parquet

Staging is the first Spark step. It reads the Bronze XML with Spark 4.2's XML reader [1] and a **fixed schema**, so IDs and times are kept as text. This way leading zeros and times like `25:10:00` stay exactly as they are. We use `FAILFAST` mode, so a broken record stops the job instead of turning into an empty row. Parts of the XML that are not in our schema are saved in `_unmapped_xml`.

```
def reader(tag, schema, files):
    # FAILFAST: a malformed XML record stops the job instead of becoming a null row.
    return (spark.read.format("xml").option("rowTag", tag)
            .option("mode", "FAILFAST")
            .option("wildcardColName", "_unmapped_xml")
            .schema(schema).load(files))

raw = reader("CompositeFrame", complete_frame_schema(), xml_files)
```

Each `CompositeFrame` is joined with a list of the source files (operator, checksum, dates) and then split with `posexplode` into one table per NeTEx type. Every row keeps its frame IDs and position, which we use later to build the Silver keys.

We write 16 staging tables as Parquet: **definitions** (`frames`, `lines`, `routes`, `journey_patterns`, `journey_pattern_points`, `service_journeys`, `passing_times`), **stops** (`scheduled_stop_points`, `stop_assignments`, `flexible_stop_places`, `stop_places`, `quays`) and **calendars** (`day_types`, `operating_periods`, `operating_days`, `day_type_assignments`).

For every table, the number of rows has to be the same as in Bronze, and every row needs an ID. The Parquet file is written, read back and counted again, and its schema and size are saved in the manifest. At the end we check the XML checksums again to make sure Spark did not change the source.

3.3 Silver Layer: Cleaning, Standardisation and Enrichment

Silver is the most important part of our project. The source data is mostly correct, so Silver is less about deleting bad rows and more about **connecting the references, working out the dates and converting the times correctly, and proving that no rows were lost or doubled**.

3.3.1 Silver build order

1. Read the staging Parquet and give every row a SHA-256 **occurrence key** based on where it comes from.
2. Write the tables for lines, routes, journey patterns and the stops in each pattern.

3. Merge stop definitions that are exactly the same, and quarantine definitions that have the same key but different content.
4. Expand the calendars into real dates (`calendar_dates`) and save any problems (`calendar_issues`).
5. Mark each journey as fixed or on-demand and find its day types (`journey_day_types`).
6. Connect every passing time to its stop, convert the times and add reason codes (`passing_times`, `quarantine_passing_times`).
7. Connect every journey to its line, pattern, route and calendar, and add reason codes (`journeys`, `quarantine_journeys`).
8. Join journeys with their dates to get one row per journey per day (`journey_service_dates`).
9. Check that `lines`, `journeys` and `passing_times` have exactly the same number of rows as in staging, and save the quality numbers in the manifest.

Every table is saved, read back and counted. The last check makes sure that no rows were lost or doubled by the joins:

```
if (report["tables"]["lines"]["rows"]        != input_counts["lines"] or
    report["tables"]["journeys"]["rows"]     != input_counts["service_journeys"] or
    report["tables"]["passing_times"]["rows"] != input_counts["passing_times"]):
    raise RuntimeError("Silver joins changed source occurrence counts")
```

Every row gets its own key: a SHA-256 hash of the snapshot, operator, source file, frame and position. This means that the same ID and version in two different files is **not** treated as a duplicate. We needed this because AKT really does repeat IDs in different files.

3.3.2 Data model and reference resolution

To know where a bus stops, we have to follow a chain of references: passing time → pattern point → scheduled stop → stop assignment → quay → national stop place (or a flexible area). A journey also links to its pattern, route and line. All these joins use one function, `lookup`, which counts how many matches each row has and only accepts **exactly one**. No match is "unresolved" and more than one is "ambiguous". We never guess.

```
result = source.join(target, condition, "left")
window = Window.partitionBy("occurrence_key")
result = result.withColumn(prefix + "matches",
                           F.count(prefix + "occurrence_key").over(window))
result = result.withColumn("_candidate_row", F.row_number().over(
    window.orderBy(F.col(prefix + "occurrence_key").asc_nulls_last())))
return (result.where("_candidate_row=1").drop("_candidate_row")
        .withColumn(prefix + "resolved", F.col(prefix + "matches") == 1))
```

The join follows the NeTEx rules: the ID must match, the version must match (unless it is empty or `any`), the operator must match, and local references must be found in the same file.

3.3.3 Cleaning: keep, flag and quarantine

We do not delete any rows. Each row gets a list of reason codes, and `is_valid` is true only if the list is empty. Bad rows are also copied to `quarantine_*` tables so they are easy to look at.

```
reasons = [
  (~F.col("point_resolved"),      "unresolved_pattern_point"),
  (~F.col("stop_resolved"),       "unresolved_scheduled_stop"),
  (~F.col("assignment_resolved"), "unresolved_stop_assignment"),
  (~(F.col("quay_resolved") | F.col("flex_resolved")), "unresolved_physical_stop"),
  (invalid_time, "invalid_time"),
  (F.col("departure_seconds") < F.col("arrival_seconds"), "arrival_after_departure"),
  (F.coalesce("arrival_seconds", "departure_seconds")
       < F.col("previous_seconds"), "non_monotonic_time")]
times = times.withColumn("reason_codes", F.filter(
    F.array(*[F.when(c, F.lit(r)) for c, r in reasons]),
    lambda x: x.isNotNull()))
```

Stop definitions that are exactly the same are merged. If two definitions have the same key but different content, both are held back and quarantined. Journeys can also get these reasons: `unresolved_line`, `unresolved_pattern`, `unresolved_route`, `unresolved_calendar`, `invalid_passing_times` and `missing_passing_times`.

3.3.4 Standardisation of times

We keep the original time text and also convert it to seconds from the start of the operating day, including the day offset. So `00:10:00` with offset 1 becomes 87,000 seconds, and a trip that goes past midnight stays on the same day. Then a `lag` window checks that the times in each journey never go backwards.

3.3.5 Enrichment: calendars and service dates

The calendar tables are small, so we collect them to the driver and expand them in normal Python (`silver/calendars.py`). We handle weekdays, date periods and single dates. A single date always wins over a period, and if a date is both added and removed, it is removed. Rules we do not support, like holiday types, are saved as calendar problems instead of being treated as normal days. The dates are then sent back to Spark and joined with the journeys.

By default we use the whole month of the snapshot. On-demand HentMeg trips keep their time window, but they are marked separately and never counted as fixed departures.

3.3.6 Quality evidence: before and after

`pipeline/quality/silver_profile.py` checks every Silver output per operator. "Before" is staging (all source rows) and "after" is Silver. We profiled ten snapshots: the nine historical ones (15 January–15 September) and the first scheduled snapshot from 8 November 2026. The table shows two of them for each operator, using the seven-day window that starts on the snapshot date.

Measure	AKT 15 Jan	AKT 15 Jul	KOL 15 Jan	KOL 15 Jul
Lines: staging → Silver	212 → 212	207 → 207	325 → 325	323 → 323
Journeys: staging → accepted	8,845 → 8,845	10,535 → 10,535	16,444 → 16,444	15,046 → 15,046
Passing times: staging → accepted	277,159 → 277,159	343,316 → 343,316	491,542 → 491,542	422,108 → 422,108
Quarantined journeys / passing times	0 / 0	0 / 0	0 / 0	0 / 0
Unresolved / ambiguous references	0 / 0	0 / 0	0 / 0	0 / 0
Overnight passing times kept with offset	6,289	9,558	31,607	24,470
Service occurrences in window	21,236	15,245	32,109	25,295
→ excluded as flexible (on demand)	0	0	23	13
Valid fixed journeys not running in window	1,715 of 8,845	5,364 of 10,535	6,063 of 16,432	5,900 of 15,036
Optional fields missing (stop name / direction)	277,159 / 0	343,316 / 0	24 / 2	20 / 2
Publication gate	pass	pass	pass	pass

Table 1. Silver quality evidence (from docs/evidence/quality_report.md)

Note on missing stop names: the 277,159 and 343,316 missing values for AKT are an optional field, not rejected rows. AKT does not give names on its scheduled stop points, so every AKT passing time has an empty stop name there. We get the stop names from the national stop register (`NSR:StopPlace`) instead, so nothing is lost.

In all ten snapshots (nine historical plus 8 November) nothing was rejected, merged or unresolved. The source data is consistent, and our checks show that Silver did not lose or double any rows. We still learned some useful things: many trips run after midnight (up to 52,210 Kolumbus passing times in one snapshot), many trips do not run in a given week (for example school buses in holidays), and AKT stops have no names, so we take the names from the national stop register.

At first we thought `ServiceJourney.DepartureTime` was required. But almost all journeys leave it empty, because the Nordic NeTEx profile puts the times on the passing times. So we changed it to optional.

Independent source checks.

`pipeline/quality/trace.py` reads the original XML again with only normal Python (no Spark and no Silver code). It works out the times, dates and stops by itself and counts every trip of the chosen line. We use it on **four representative lines**, one for each difficult case (normal, overnight, calendar exception and on-demand), on the full 15 January snapshot. All four passed:

This trace does not check every journey in every snapshot. The broad evidence for all data is the row-count check between staging and Silver, the quality profile above and the exact match between Gold and Silver. The trace adds an independent check of the logic on the hardest cases.

Case	Line	Passing times checked	Source journey occurrences	Fixed departures in window	Result
Ordinary	AKT 411	14	20	52	pass
Overnight	AKT 100	69	150	418	pass
Calendar exception	KOL SK2104	9	11	14	pass
Flexible	KOL HentMeg	2	7	0 (11 flexible excluded)	pass

Table 2. Independent XML-to-Gold traces, 15 January 2026

3.3.7 Implementation: Spark APIs used

- **XML reader** in Spark 4.2 with a fixed schema for each NeTEx type, saving to Parquet.
- **DataFrame API**: `select`, `withColumn`, `join`, `groupBy().agg`, `dropDuplicates`, `posexplode`, and `F.filter` on arrays for the reason codes.
- **Window functions**: `count` and `row_number` to find matches, and `lag` to check that times go forward.
- **Spark SQL expressions** inside the DataFrame code, for example `where("is_valid AND NOT is_flexible")`.
- **Actions**: every table is written, read back and counted, and the count is saved in the manifest. Silver then continues from the saved table, which also keeps the Spark plans short.

We did not use RDDs. The DataFrame API was a better fit for our joins.

3.3.8 Performance

- The calendar tables are small, so we handle them on the driver. The big tables stay in Spark.
- We set `spark.sql.shuffle.partitions` to 4, because the server only has 2 cores.
- We cache tables on disk and clear the cache straight after use, to save memory.
- We turned off broadcast joins in route history, because they made Java run out of memory.
- Each snapshot runs in a new Spark process with 2 GiB of memory, because one long process crashed after 14 of 22 jobs.

3.3.9 Lifecycle: testing, versioning and reruns

Each step's checkpoint includes a checksum of its code and its input. If we change the Silver code, Silver runs again in a new folder, but the steps before it are reused. A step only counts as `complete` when its manifest says so, and before reusing it we check the saved Parquet files. Failed runs are never deleted, and a lock stops two processes from working on the same snapshot. On the server, running the whole history again reused all 27 finished steps and created no new folders.

3.4 Gold Layer: Serving the Outcome

Gold answers the user's questions: how many departures each line and each operator has per day, what that adds up to per month, which stops are used, and what was left out and why. Gold only uses **valid, fixed** trips from Silver, and is only built when the quality gate passes.

3.4.1 Gold build order

1. Check Silver with the quality gate (Section 3.4.5). If it **fails**, stop and mark the run as failed.
2. Take the trips we can count: `journey_service_dates` where `is_valid AND NOT is_flexible`.
3. Count departures per line per day (`daily_line_service`), and **check** that the total matches Silver.
4. Make a grid of every operator and every date, and add the daily totals to it (`daily_network_service`).
5. Summarise each line for the whole period (`line_window_summary`).
6. List the stops used by trips that actually run (`line_stop_coverage`).
7. Save every trip we did not count, with the reason (`excluded_service_dates`), and write the Gold manifest with the quality results.

3.4.2 Gold tables and columns

Table	Grain	Main columns
`daily_line_service`	line × operating date	provider, line ID/version, public code, mode, `service_date`, `scheduled_departures`, first/last departure seconds
`daily_network_service`	provider × calendar date	`scheduled_departures`, `active_lines`, first/last departure, `is_source_covered`, `snapshot_date`
`line_window_summary`	line × window	`scheduled_departures`, `days_with_service`, `window_days`, `average_departures_per_calendar_day`, earliest/latest departure
`line_stop_coverage`	line × stop	scheduled stop reference, quay or flexible stop place reference
`excluded_service_dates`	journey × date	every occurrence that is invalid or flexible, with its flags

Table 3. Gold tables

Gold is different from Silver on purpose. Silver keeps all the detailed rows and where they came from. Gold is small and made for one use, so the GUI can read it with pandas without starting Spark.

3.4.3 Correctness: reconciliation

```
daily = valid.groupBy(*keys, "service_date").agg(
    F.count("*").alias("scheduled_departures"),
    F.min("first_departure_seconds").alias("first_departure_seconds"),
    F.max("first_departure_seconds").alias("last_departure_seconds"))
daily = write("daily_line_service", daily)
reconciled = daily.agg(F.coalesce(F.sum("scheduled_departures"), F.lit(0))
                       .alias("n")).first().n
if reconciled != eligible:
    raise RuntimeError("Gold departures do not reconcile "
                       "to eligible Silver service dates")
```

If even one departure is missing or counted twice, the build fails. This check passed for every snapshot we published, for example 466,493 departures across the nine historical weeks.

3.4.4 Zero versus unknown: the network date spine

A day with no buses is not the same as a day with no data. So Gold makes a full grid of every operator and every date (`spark.range` + `crossJoin`) and joins the daily totals onto it. If a date is inside the operator's calendar but has no trips, it becomes **0**. If it is outside the calendar, it stays **empty** and `is_source_covered` is false.

Averages are divided by **all** days in the period, including days with no buses, so a holiday week lowers the average instead of being hidden.

3.4.5 Publication gate

Matching Silver is not enough on its own, because Gold would also match a Silver table that lost a lot of rows. So before writing any table, `build_gold` checks Silver against `config/quality_thresholds.json`:

Rule	Warn when	Stop when
Provider selection	–	No declared providers, an expected provider missing, or an undeclared one present
Staging → Silver occurrence counts	–	Any difference
Journeys / passing times quarantined	any	> 1%
Service occurrences rejected as invalid	any	> 1%

Rule	Warn when	Stop when
Essential fields missing	any	> 1%
Ambiguous references	–	any
Unresolved references, conflicting definitions, calendar issues	any	–
Journeys affected by unresolved calendars	any	> 1% (counted before date expansion)
Accepted service occurrences	–	none for a provider

Table 4. Quality gate rules

pass means Gold is published. **warn** means it is published but the problems are saved in the manifest. **fail** means Gold is not published, the run is marked failed and the GitHub step fails. The quality results are saved in `metadata/quality_profile.json` next to Gold. We chose the limits after checking all ten 2026 snapshots: none had any rejected rows, so any rejection should be noticed, and more than 1% means something is seriously wrong. Our test `test_invalid_times_are_quarantined_and_the_quality_gate_stops_gold` breaks half of the AKT journeys and checks that Gold is stopped.

3.4.6 Full-month reconstruction

Each export covers a certain period and exports overlap. To get one answer per date, the GUI uses the **newest export that covers that date for both operators**. We always use whole exports. We never add two exports together, and we never fill in missing lines from an older export. The source export date is included in every daily download.

On the school server the rebuild `monthly-67b04d35cf56` finished all 22 jobs and covered January–November 2026. Our detailed check of January–September covers all 273 days (546 operator/day pairs), all 18 gates passed, and there are 1,918,707 planned departures (Section 4.2).

3.4.7 Gold in use: the real November 2026 overview

The screenshot below shows our GUI with the real Kolumbus and Agder data. All the numbers on the cards come from the Gold table `daily_network_service`:

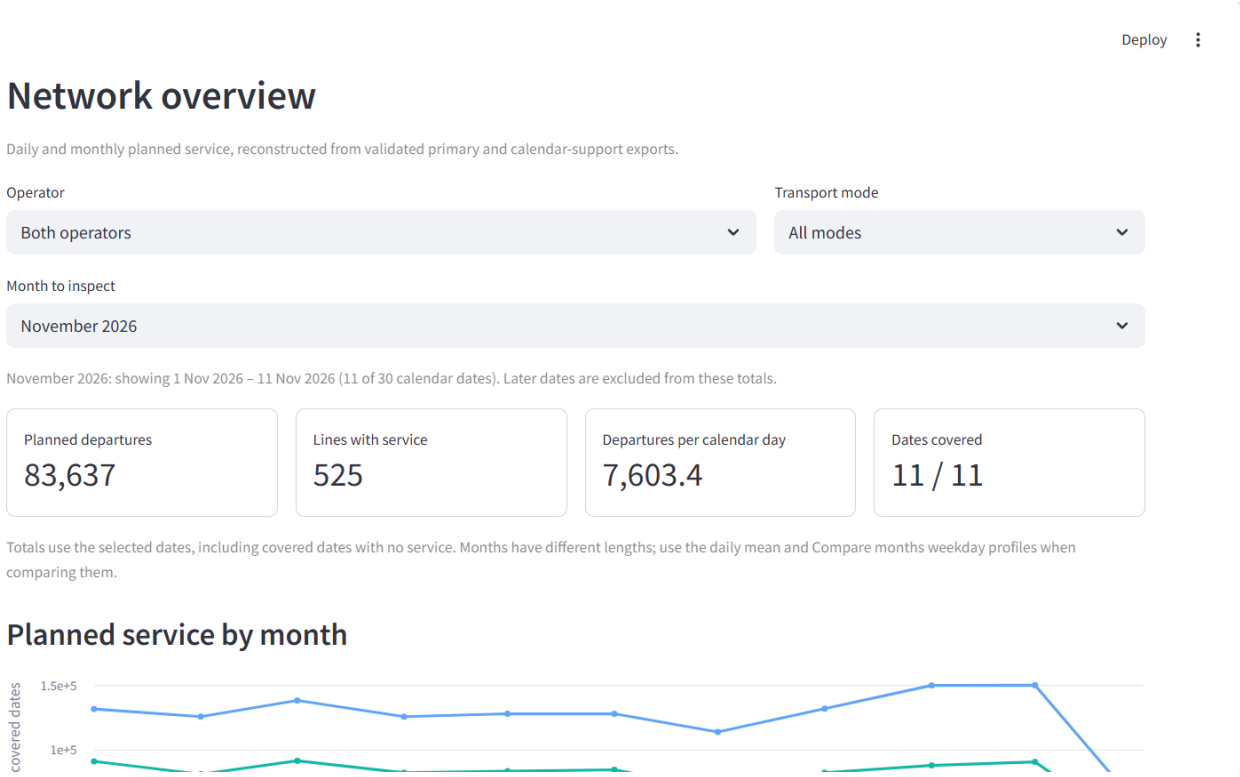


Figure 2. GUI Overview with the real data, November 2026 (1–11 Nov): 83,637 planned departures, 525 lines with service, 7,603.4 departures per calendar day, 11 of 11 dates covered

- **Planned departures 83,637** is the total for both operators and all modes from 1 to 11 November. Later dates are not included because the GUI stops at today's date (11 November 2026).
- **Departures per calendar day 7,603.4** is 83,637 ÷ 11 days, counting every day, also days with no buses.
- **Dates covered 11 / 11** means we have data for every one of the 11 days.
- **Planned service by month** shows the total for each month and operator. The last point is low only because November is not finished yet.

3.5 Route History and Verified Changes

`pipeline/analysis/route_history.py` compares each snapshot with the next one, line by line. A line is identified by its operator and NeTEx line ID. We compare national stop places, so moving to another platform at the same stop is not a route change. Line versions use **SCD Type 2**:

a new version is only made when the bus number, name, mode or stops change. A change in the number of departures is saved as a change event, but does not make a new version.

- If a line is missing in the next export, we only say it "disappeared". We only call it `renumbered_verified` or `discontinued_verified` if our files `config/route_renumberings.json` or `route_discontinuations.json` have a matching entry with the operator's link and quote. If an entry does not match any change, the run fails.
- To find possible renumberings we compare the stops of the old and new line (Jaccard similarity). The real data taught us two things: operators publish the new line ID one month early, and bus numbers are reused, so lines with the same number need at least 0.2 similarity.
- We also check that: lines after = lines before + added − removed, every Gold date links to exactly one line version, and the departures add up to the Gold total.

3.5.1 Results

The 9 November run used ten snapshots: the nine historical ones plus the first scheduled snapshot from 8 November 2026. It passed all checks. 521,185 Gold departures (476,651 bus and 44,534 boat) match the history tables. 5,358 line snapshots give 1,336 line versions. We found 31 possible renumberings. 16 renumberings and 4 removed lines are confirmed by the operators, and all 20 match the data. 24 lines that disappeared have no confirmation, so we only say they are no longer published.

3.5.2 Case study: Kolumbus Bussveien, 1 July 2026

Operator notice	Old line	New line	Similarity	How found
Rute 1 Stavanger – Hundvåg blir nå rute 9	`8_1000` (1)	`8_1109` (9)	1.00	similar_stops
Rute 4 Rosenli – Madlakrossen blir nå rute 12	`8_1005` (4)	`8_1112` (12)	1.00	similar_stops
Rute 2 Stavanger – Sandnes blir nå rute 1	`8_1002` (2)	`8_1001` (1)	0.26	verified_mapping
rute 2 Stavanger – Risavika blir rute 4	`8_1002` (2)	`8_1104` (4)	0.51	verified_mapping
Rute 3 Stavanger – Forus blir nå rute 2	`8_1004` (3)	`8_1003` (2)	0.46	verified_mapping

Table 5. Verified Kolumbus renumberings (source: Kolumbus notice [6])

Old line 2 was split into new lines 1 and 4. Our rules also suggested `8_1002 → 8_1003` because both had number 2 (similarity 0.22), but the notice says something different, so we did not accept it. This shows that the same bus number alone is not enough proof.

3.5.3 Case study: Agder, 22 June and 1 July 2026

AKT's notice about route changes from 22 June and 1 July 2026 [7] matches our data. Route 56 (`AKT:Line:1056`, 178 departures a week) was removed. Bus 55 moved from `AKT:Line:1055` to `AKT:Line:55`: the new line was published on 15 June with no departures and had all 277 departures on 15 July, and it was extended to Odderhei (25 → 28 stops). Eight stops that only line 56 used lost all their buses. Route 178 was replaced by 172, but the stops are only 0.46 similar, so we linked them using the confirmed mapping.

3.5.4 Explaining the February drop

AKT's weekday departures drop by about 25% in 15–21 February, while weekends stay the same, and only 116 of 212 lines run. This is the Agder winter school holiday (week 8, 16–20 February 2026), not a route change. AKT lists 3,744 stops in its routes that week, but only 2,269 are actually used.

3.6 Automation, Monitoring and GUI

3.6.1 Automation with GitHub Actions

Our workflow `.github/workflows/ingest-netex.yml` runs on every push to `main`, when we start it by hand, and on a schedule on days 1, 4, 7 and so on of each month (`23 8 */3 * *`). The schedule starts again at the beginning of each month, so the gap between two runs is not always exactly 72 hours. It runs on our own runner on the school server, so the data stays on the server's disk. The workflow runs the tests, downloads the data, processes it through Silver and Gold, updates route history and restarts the GUI. If one step fails, the next steps do not run.

Runs we checked: a manual run 40388358483 (26 October), a push run 40537981353 (27 October), and three scheduled runs on 27–28 October. Each took about 53 seconds for ingestion on `dat535-spark`. GitHub started the scheduled runs hours later than we asked, so we wrote down what actually happened.

3.6.2 Environments and configuration

- When developing we write to the `data/` folder, which is ignored by Git. On the server we use `/srv/dat535/data`, set in the GitHub variable `DAT535_DATA_ROOT`.
- Spark and the GUI have their own Python environments (`~/spark-env`, `~/dat535-gui-env`). Spark uses its own Java 17, so the server's default Java 8 is not changed.
- All rules are in config files in Git: the quality limits and the route-change evidence. No passwords are needed.

3.6.3 Monitoring and visibility

We can see the status, errors, quarantined rows and quality results in four places: the GitHub Actions log and report, the JSON manifest of each step, the checkpoint database (`python3 -m pipeline.checkpoints`) and the Data checks page in the GUI.

3.6.4 GUI

The Streamlit app only reads data. It finds the finished Gold outputs through the checkpoint database, reads

small Parquet tables with pandas and never starts Spark. It has these pages: Overview, Daily service, Compare months, One line, What changed, Replaced lines and Data checks. By default it shows dates up to today, but future plans can be turned on. Section 3.4.7 shows the Overview page with the real data.

The GUI is deployed on the school server. `scripts/run_gui.sh` starts it in the background on all network interfaces (`0.0.0.0:8501`), so it can be opened directly at `http://152.94.163.144:8501` when the firewall allows port 8501, or through SSH port forwarding at `http://localhost:8501`. The workflow restarts it after every successful run, so it always shows the newest Gold data.

4 Evaluation

In this section we look at the results on the real 2026 data: the historical snapshots, the full months and the run times. The Silver quality results are in Section 3.3.6 and the route-history results in Section 3.5.

4.1 Processing of the nine historical snapshots

Snapshot	Journey occurrences	Passing-time occurrences	Gold fixed departures (7 days)
2026-01-15	25,289	768,701	53,322
2026-02-15	28,628	906,396	48,927
2026-03-15	32,702	1,034,292	53,461
2026-04-15	32,974	1,075,434	53,369
2026-05-15	34,328	1,109,078	53,580
2026-06-15	34,089	1,092,197	52,936
2026-07-15	25,581	765,424	40,527
2026-08-15	23,592	725,385	54,904
2026-09-15	26,574	820,047	55,467
Total	**263,757**	**8,296,954**	**466,493**

Table 6. Server run of 28 October 2026 on group4-spark-vm (all stages completed)

4.2 Full-month planned service

Month	Dates covered	AKT	KOL	Total
2026-01	31/31	90,368	131,583	221,951
2026-02	28/28	80,266	125,589	205,855
2026-03	31/31	90,803	138,292	229,095
2026-04	30/30	81,428	125,517	206,945
2026-05	31/31	82,645	127,846	210,491
2026-06	30/30	83,767	127,770	211,537
2026-07	31/31	68,712	113,513	182,225
2026-08	31/31	81,499	131,815	213,314
2026-09	30/30	87,204	150,090	237,294
Total	**273/273**	**746,692**	**1,172,015**	**1,918,707**

Table 7. Reconstructed planned fixed departures, bus and water (local verification, 10 November 2026)

July is the lowest month for both operators because of the summer timetables. September is the highest month for Kolumbus, which fits with the new Bussveien network running for a full month after the summer. Months have different numbers of days and weekdays, so the GUI also shows daily averages and weekday profiles.

Why the totals are different. The report uses four departure totals, and each one counts a different period. **466,493** is the sum of the seven-day windows of the nine historical snapshots (Table 6). **521,185** in route history is the same nine windows plus the seven-day window of the 8 November snapshot (54,692 departures). **1,918,707** is the full months of January–September, rebuilt from the nine historical exports and nine calendar-support exports (Table 7). **83,637** in the GUI is 1–11 November 2026 only. They are not supposed to add up to each other.

4.3 Execution Times

The table below shows the run times we measured on the school server.

Stage	Run	Data	Time
Ingestion (manual CLI)	24 Oct, school server	23 Oct export, 532 XML files	43.5 s
Ingestion job (Actions)	dispatch 26 Oct; push 27 Oct	one national export	52 s each
Scheduled ingestion jobs	3 runs, 27–28 Oct	one national export	≈ 53 s each
Ingestion step	Actions run, 10 Nov	one national export	43 s
Spark test suite step	Actions run, 10 Nov	test fixtures	3 min 36 s
Snapshot processing (staging → Gold)	Actions run, 10 Nov	one snapshot	28 s
Route history (failed, later fixed)	Actions run, 10 Nov	all snapshots	1 min 47 s

Table 8. Measured execution times

Ingesting one export takes less than a minute, so running on days 1, 4, 7 and so on of each month is easy for the server. In the Actions run, the tests took longer than the processing itself. Our real problems were memory and disk space (Section 6), not speed. When we ran the whole historical command a second time, it reused all 27 finished steps and did not download or build anything again.

4.4 Tests and Run Evidence

Our test suite has 175 tests in 18 test files, with both normal unit tests and Spark tests that run the real transformations on small NeTEx test files. The GitHub Actions workflow runs the ingestion tests before every download and the Spark tests before every processing run on the school server, so the pipeline only runs when the tests pass. In our last full test run recorded in our documentation (10 November 2026), all tests at that time passed with Spark and the GUI libraries.

The tests cover things like quarantining bad times, stopping Gold when the quality gate fails, calendar rules, duplicate uploads, broken ZIP files, checkpoint reuse and the GUI calculations. For example, `test_invalid_times_are_quarantined_and_the_quality_gate_stops_gold` breaks half of the AKT journeys and checks that Gold is not published.

We also tested the checkpoints on the real data. On 28 October we ran the whole historical command on the server a second time. It reused all 27 finished steps, the checkpoint registry still had exactly 27 attempts, and no new folders were created.

4.5 Summary

From the evaluation we can say that:

- Ingestion runs by itself, checks every file and stops with a clear error instead of guessing a date.
- Silver kept all 263,757 journeys and 8,296,954 passing times from the historical data. No rows were quarantined, and our independent XML checks matched Gold for all four representative cases.
- Gold always matches Silver and is only published if the quality gate passes. It gives 1,918,707 planned departures for all 273 days of January–September.
- Route history finds real changes in the network, and only calls something a renumbering when the operator has confirmed it.
- Everything can be run again: the tests run before every pipeline run on the server, and a second run reuses the saved checkpoints.

5 Reproducibility

The project is a normal Python project that we run from the command line. We did not use notebooks. Every step writes a JSON manifest, and every Spark step is saved in a checkpoint registry, so someone else can run it again and check the results.

5.1 Folder Structure

```
dat535-project/
  app/              Streamlit GUI
  config/           gate thresholds and
                    route-change evidence
  pipeline/
    ingestion/      poll, manual ZIP,
                    historical, support
    processing/
      bronze/       XML -> staging Parquet
      silver/       build.py, calendars.py
      gold/         build.py
    quality/        profile, trace, report
    analysis/       route history
    storage/        ZIP Bronze, maintenance
    run_snapshot.py   staging->Silver->Gold
    checkpoints.py    checkpoint registry
  scripts/          run.sh, run_gui.sh
  tests/            175 unit and Spark tests
  .github/workflows/ingest-netex.yml
```

5.2 Execution Steps

1. **Set up the environments.** Install Python 3.11 or newer and Java 17. Create `~/spark-env` and `~/dat535-gui-env` and install `requirements.txt` in both.
2. **Ingest data.** Run the command below to get yesterday's export into Bronze. The GitHub Actions workflow also does this automatically.

```
python3 -B -u -m \
  pipeline.ingestion.poll_entur_netex \
  --zip-only \
  --remove-national-after-validation
```

3. **Process.** Run `bash scripts/run.sh processing --manifest <manifest.json> --data-root /srv/dat535/data` to build staging, Silver and Gold.
4. **Build route history.** Run `bash scripts/run.sh history --data-root /srv/dat535/data`.
5. **Open the GUI.** Run `bash scripts/run_gui.sh` and open port 8501.
6. **Check the results.** `bash scripts/run.sh quality-report` makes the quality report, `bash scripts/run.sh tests -v` runs the tests, and `python3 -m pipeline.checkpoints` shows every run.

6 Challenges Faced

Most of our problems came from the real data and from the small school server, not from Spark.

1. Exports only cover part of a month. At first we only used the 15th–21st of each month. When we changed to full months, we found that a mid-month export does not include the first days of the month. Most 15 January files only start on 12–13 January. We fixed this by adding extra exports from the start of each month, and a check that tells us if any day is missing.

2. Broken exports. The exports from 1 and 2 February 2026 had a missing stop reference (`NSR:Quay:43289`). We did not try to repair them. We kept them and used the 14 January export instead, because it already covers February.

3. The same ID in different files. AKT sometimes uses the same ID and version in more than one file. If we had

treated ID + version as unique, rows would have been merged or doubled. So we gave every Silver row its own key based on where it comes from in the source.

4. Not enough disk space. The school VM only has 38 GiB. The unzipped XML filled Bronze up to 23 GiB and left only 3.5 GiB free. Removing duplicate files helped for a while, but the real fix was to keep the XML inside ZIP files and unzip one snapshot at a time. After that we had 23 GiB free.

5. Java ran out of memory. Route history crashed while Spark tried to broadcast a big table, and the monthly rebuild crashed after 14 of 22 jobs. We turned off automatic broadcast joins in route history, cleared the cache after each table, and started a new Spark process with 2 GiB of memory for each snapshot.

6. Times after midnight and on-demand buses. Some times are written as 00:10 plus one day, and HentMeg buses have a time window instead of a fixed time. We changed all times to seconds from the start of the operating day, and we kept on-demand trips but left them out of the departure counts.

7. The quality gate could pass with no data. Our first gate only looked at the operators it found in Silver, so a missing operator would not be noticed. In version 2 the gate compares Silver with the operators we expected from the manifest.

8. GitHub scheduling was not on time. The scheduled runs started hours later than we asked for. We save the trigger type in every manifest and do not depend on the exact time.

9. Is it really a renumbering? Bus numbers are reused, and operators publish new line IDs a month early. Similar stops only give us a guess, so we only accept a renumbering if the operator confirms it. For example, the guess that line 2 became line 2 in Stavanger was wrong.

7 Alternatives

There were other ways we could have built the project. Here we list some of them and what they would change.

1. GTFS instead of NeTEx. Entur also publishes timetables in GTFS, which is simpler to read. We chose NeTEx because it has line versions, calendar rules and on-demand services, which we needed for route history.

2. Delta Lake or Apache Iceberg. These table formats have safe writes and time travel built in. They could replace part of our own checkpoint system, but they would add another tool to a small server.

3. A workflow tool like Airflow. Airflow or Dagster would give us scheduling, retries and run history out of the box. We used GitHub Actions because it was easier to set up on the school server and it runs our tests in the same place.

4. A ready-made NeTEx validator. A full NeTEx validator could find more problems in the source files than our own reference checks.

5. Cloud storage for Bronze. Keeping Bronze in cloud storage would remove the disk space problem that made us switch to ZIP files.

6. A database for Gold. Putting Gold into PostgreSQL or DuckDB would let people query it with SQL. Right now the GUI reads the Parquet files directly.

7. Collecting data every day. Getting an export every day instead of on days 1, 4, 7 and so on of each month would show changes on the exact day, but needs about three times the storage.

8 Future Work

1. Upload page. Add an upload page to the GUI and use the same duplicate check for manual and scheduled ingestion.

2. More data. Collect every daily export and add more operators than Kolumbus and Agder.

3. Plans compared to reality. Add Entur's real-time data to compare the planned timetable with what actually happened.

4. Better performance measuring. Save the time of each Spark stage from the Spark UI so we can tune the pipeline using real numbers.

5. More checks and time zones. Add full NeTEx validation, more holiday calendar rules and proper time zone handling.

6. Running on a cluster. Run the same Spark code on a cluster with several machines to test how it scales.

9 Lessons Learned

Looking back at our own work from a technology point of view:

What worked. Keeping every row and marking problems with reason codes, instead of deleting them, made it easy to prove that no data was lost. Saving checkpoints with a checksum of the code and input made re-runs safe and fast. Using the high-level DataFrame API with Spark SQL expressions kept the Silver code readable.

What did not work at first. Optimising by guessing did not help. Our real problems were memory and disk space, and we only found them by running on the real data on the school server (Section 6). A seven-day window was also too small to describe a month.

Why we made our main decisions. We chose NeTEx over GTFS because route history needs line versions and calendar rules. We kept Bronze as checked ZIP files because of the small disk. We added a quality gate because matching Silver alone does not prove that Silver is complete.

What we would improve. Measure Spark stage times systematically, collect data every day, and add an upload page to the GUI (Section 8).

10 Conclusion

We built a Bronze → Silver → Gold pipeline for the Kolumbus and Agder timetables from Entur. Ingestion

runs automatically on the school server, checks every file and keeps the raw data unchanged. The Silver layer connects the NeTEx references, converts the times and expands the calendars without losing or doubling any rows, and we checked this with row counts, the Gold–Silver match and an independent XML check of four representative lines. Gold is only published when the quality gate passes and always matches Silver. It gives 1,918,707 planned departures for January–September 2026. Route history finds real changes, like the Kolumbus Bussveien changes and the Agder route changes, and connects them to the operators' own notices.

What worked best for us was keeping all the data instead of deleting it, saving checkpoints so we could re-run safely, and only claiming something when we had proof. The biggest lessons were to measure before trying to optimise, and to be careful about what the data really shows.

References

[1] Apache Software Foundation. Apache Spark 4.2.0 Documentation. https://spark.apache.org/docs/4.2.0/. Accessed November 2026.

[2] Entur. NeTEx timetable data and Nordic profile documentation. https://entur.atlassian.net/wiki/spaces/PUBLIC. Accessed November 2026.

[3] Norwegian Digitalisation Agency. Norwegian Licence for Open Government Data (NLOD). https://data.norge.no/nlod/en. Accessed November 2026.

[4] Databricks. Medallion Architecture. https://docs.databricks.com/en/lakehouse/medallion.html. Accessed November 2026.

[5] GitHub. GitHub Actions documentation: self-hosted runners and scheduled workflows. https://docs.github.com/en/actions. Accessed November 2026.

[6] Kolumbus. Ruteendringer 1. juli 2026. https://www.kolumbus.no/aktuelt/ruteendringer-1.-juli-2026/. Accessed November 2026.

[7] AKT. Route changes in Agder from 22 June and 1 July 2026. https://www.akt.no/english/news/route-changes-in-agder-from-22-june-and-1-july-2026.9944.aspx. Accessed November 2026.